# Universal IoT Gateway

## Software Architecture & Implementation Roadmap

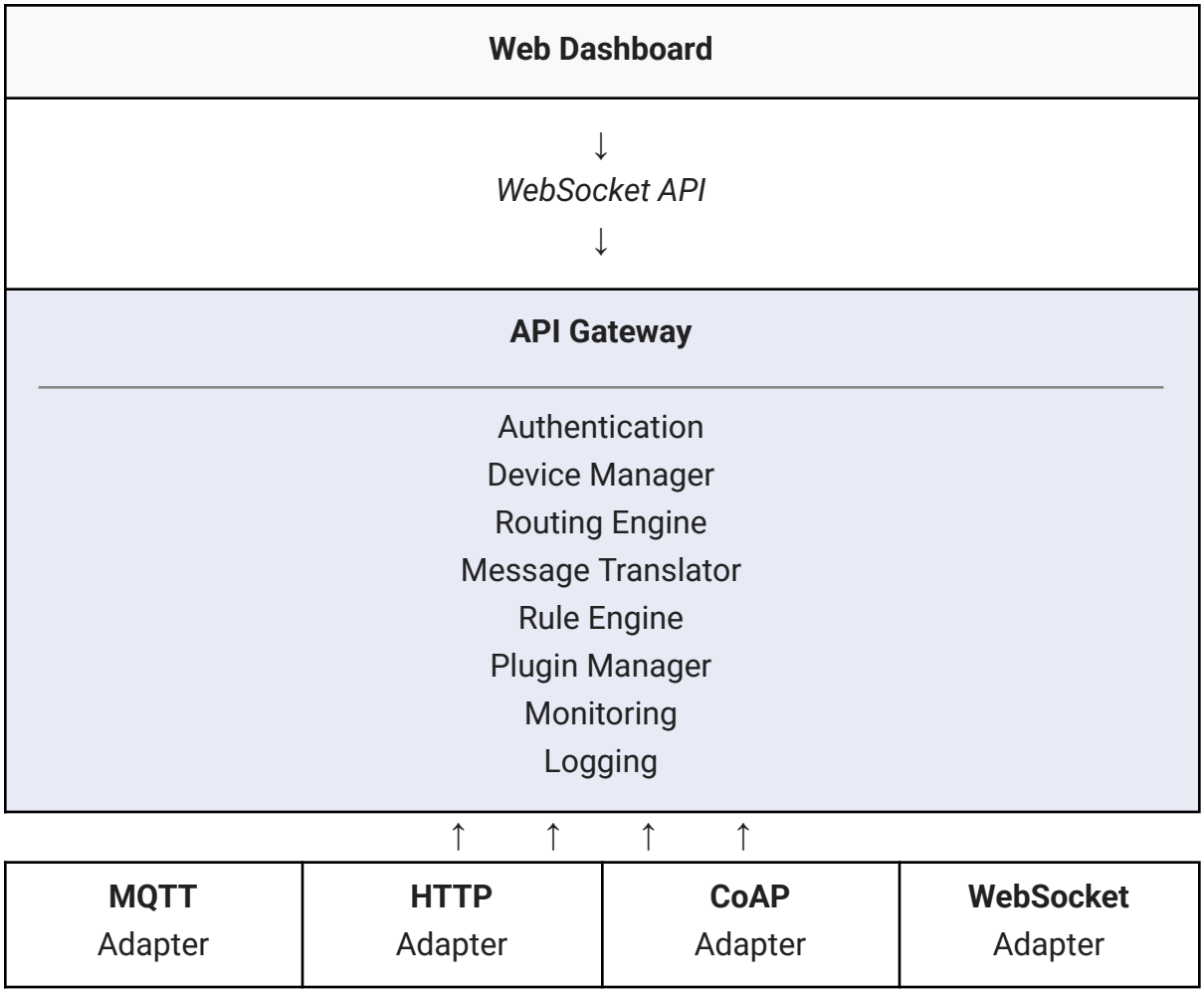
# Table of Contents


1. Final Vision
2. Development Roadmap
    - Phase 1: Design the Architecture
    - Phase 2: Core Gateway Build
    - Phase 3: Internal Message Model
    - Phase 4: Device Manager
    - Phase 5: MQTT Adapter
    - Phase 6: HTTP Adapter
    - Phase 7: CoAP Adapter
    - Phase 8: WebSocket Adapter
    - Phase 9: Rule Engine
    - Phase 10: Dashboard
3. Suggested Project Structure
4. Internal Flow
5. Message Lifecycle
6. Database Design
7. Technologies
8. Design Aspects
9. Remaining Sections

# Final Vision

The Universal IoT Gateway is designed as a protocol-agnostic central hub. The core architectural principle is the decoupling of communication protocols from business logic. The core system remains unaware of the transport layer (MQTT, HTTP, CoAP), interacting only with a normalized internal message format.

**Universal IoT Gateway**




**Note:** The core never knows anything about MQTT, HTTP, or CoAP. It only knows "messages."

# Development Roadmap

## Phase 1 — Design the Architecture

Do not write code. Design:

- System architecture
- Components
- Data flow
- Database
- Message format
- Routing flow
- Security flow

### Deliverables:

- Architecture diagram
- Sequence diagrams
- Database ER diagram
- API specification
- Folder structure

## Phase 2 — Core Gateway Build

Build only the core.

**Core Processing Flow:**
Gateway Receive Message → Validate → Convert to Internal Message → Routing Engine → Output Queue

No MQTT. No HTTP. No CoAP. Just simulate messages.

- **Example:** Incoming JSON → Routing → Outgoing JSON

# Phase 3 — Internal Message Model

Everything becomes one object.


```
{
  "id": "uuid",

  "source": {

    "deviceId": "sensor01",

    "protocol": "mqtt"

  },

  "destination": {

    "deviceId": "dashboard01",

    "protocol": "http"

  },

  "type": "telemetry",

  "resource": "temperature",

  "payload": {

    "temperature": 40

  },

  "timestamp": "...",

  "qos": 1

}
```

Every protocol converts to this.

# Phase 4 — Device Manager

Build device registration.

**Workflow:**
Register Device → Authentication → Store Metadata → Heartbeat → Online → Offline

## Database Entities:

- Devices
- Protocols
- Sessions
- Users
- API Keys

# Phase 5 — MQTT Adapter

Now connect MQTT.

**Workflow:**
MQTT Broker → MQTT Adapter → Internal Message

## Support:

- Publish
- Subscribe
- QoS
- Retain
- Last Will

# Phase 6 — HTTP Adapter

Workflow:
HTTP POST → HTTP Adapter → Internal Message

### Support:

- GET
- POST
- PUT
- DELETE

## Phase 7 — CoAP Adapter

**Workflow:**
CoAP Packet → CoAP Parser → Internal Message

### Support:

- GET
- POST
- Observe

## Phase 8 — WebSocket Adapter

Workflow:
Dashboard → WebSocket → Internal Message

Now the dashboard becomes real-time.

## Phase 9 — Rule Engine

Instead of hardcoding routes:
MQTT → HTTP

Create rules.

- Example: IF Temperature > 40 THEN Send HTTP, Send CoAP, Send Alert.

Rules become data stored in the database.

## Phase 10 — Dashboard

Dashboard visualizes the following system metrics and data:

- Connected Devices
- Messages/sec
- CPU / Memory usage
- Logs
- Active Protocols
- Traffic Graph
- Errors
- Alerts

# Suggested Project Structure

```
None
iot-gateway/
├── apps/
│   ├── gateway-core/
│   ├── dashboard/
│   └── admin-api/
├── packages/
│   ├── core/
│   │   ├── adapters/
│   │   │   ├── mqtt/
│   │   │   ├── http/
│   │   │   ├── coap/
│   │   │   └── websocket/
│   │   ├── translator/
│   │   ├── router/
│   │   ├── rule-engine/
```

```
|   |   ├── device-manager/
|   |   ├── security/
|   |   ├── monitoring/
|   |   ├── logging/
|   |   └── plugin-system/
|   └── shared/
├── docs/
├── docker/
├── scripts/
└── tests/
```

This separation makes each module independently testable.

# Internal Flow

The heart of the system follows this sequential flow:

Device → Protocol Adapter → Validation → Authentication → Parser → Internal Message → Message Queue → Routing Engine → Rule Engine → Translator → Destination Adapter → Destination Device

# Message Lifecycle

1. Receive
2. Authenticate
3. Validate
4. Normalize
5. Log
6. Route
7. Translate
8. Deliver

9. Acknowledge
10. Store Metrics

At each step, you can hook in logging or monitoring without changing business logic.

# Database Design

## Core Tables:

- Users
- Roles
- Devices
- DeviceProtocols
- DeviceSessions
- Messages
- RoutingRules
- ProtocolMappings
- Logs
- Alerts
- ApiKeys
- Metrics

Later you can add audit trails and historical analytics.

# Technologies

| Layer | Recommendation |
| --- | --- |
| Language | TypeScript/ JavaScript |
| Runtime | Node.js |

| Layer | Recommendation |
| --- | --- |
| Framework | NestJS |
| Database | PostgreSQL |
| Cache | Redis |
| MQTT | Eclipse Mosquitto |
| WebSocket | Socket.IO |
| HTTP | NestJS Controllers |
| CoAP | node-coap |
| Queue | BullMQ or RabbitMQ |
| Dashboard | React + Vite |
| ORM | Prisma |
| Auth | JWT + Refresh Tokens |
| Containers | Docker Compose |
| Monitoring | Prometheus + Grafana (optional) |

# Design Aspects

## 1. Functional Requirements

| ID | Requirement |
|---|---|
| FR-01 | System shall register IoT devices. |
| FR-02 | System shall authenticate devices. |
| FR-03 | System shall translate MQTT messages to Internal Messages. |
| FR-04 | System shall translate Internal Messages to HTTP. |
| FR-05 | System shall support WebSocket broadcasting. |
| FR-06 | System shall execute routing rules. |
| FR-07 | System shall log every message. |
| FR-08 | System shall monitor connected devices. |
| FR-09 | System shall support plugin-based protocol adapters. |
| FR-10 | System shall provide REST APIs for management. |

## 2. Non-Functional Requirements

- **Latency:** < 100 ms
- **Capacity:** Support 1000 concurrent devices
- **Availability:** > 99% uptime
- **Extensibility:** Plugin-based architecture for new protocols
- **Security:** TLS-encrypted communication and JWT authentication

- **Scalability:** Horizontal scalability via stateless design
- **Deployability:** Containerized via Docker

## 3. Module Responsibilities

| Module Name | Purpose | Inputs | Outputs | Dependencies |
|---|---|---|---|---|
| MQTT Adapter | Receive MQTT packets | MQTT Packet | Internal Message | Translator, Routing Engine |
| Rule Engine | Execute routing rules | Internal Message | Action | Database, Translator |
| Device Manager | Manage lifecycle | Registry Data | Auth Token | Database |

## 4. Interface Contracts

TypeScript

```typescript
interface ProtocolAdapter {
  connect(): Promise<void>;
  disconnect(): Promise<void>;
  receive(data: any): Promise<void>;
  send(message: InternalMessage): Promise<void>;
  translateToInternal(data: any): InternalMessage;
  translateFromInternal(message: InternalMessage): any;
  healthCheck(): boolean;
}
```

# 5. Internal Message Specification

| Field | Type | Description |
|---|---|---|
| id | UUID | Unique message ID |
| traceId | UUID | Trace entire flow |
| protocol | String | Source protocol |
| deviceId | String | Source device |
| resource | String | Topic/URI |
| payload | Object | Actual data |
| timestamp | ISO Date | Message time |
| qos | Number | MQTT QoS level |
| metadata | Object | Additional info |

# 6. API Specification

- **POST /devices:** Register new device.
- **GET /devices:** List all devices.
- **DELETE /devices/{id}:** Remove device.
- **POST /rules:** Create routing rule.
- **GET /rules:** List rules.
- **POST /messages:** Send message.
- **GET /logs:** Fetch system logs.
- **GET /metrics:** System health metrics.
- **POST /plugins/load:** Dynamic plugin loading.

# 7. Database Schema

- **Devices:** device_id (PK), protocol_id (FK), status, last_seen, created_at.
- **RoutingRules:** rule_id (PK), condition, action, priority.
- **Relationship:** One-to-Many relationship between Users and Devices.

# 8. Configuration Management

Create a .env file with the following variables:

```shell
MQTT_HOST=localhost
MQTT_PORT=1883
POSTGRES_URL=postgresql://user:pass@localhost:5432/db
REDIS_URL=redis://localhost:6379
JWT_SECRET=super_secret_key
HTTP_PORT=3000
COAP_PORT=5683
LOG_LEVEL=debug
```

# 9. Module Dependency Matrix

| Module | Depends On | Provides To |
|---|---|---|
| MQTT Adapter | Translator | Routing Engine |
| HTTP Adapter | Translator | Routing Engine |
| Translator | Shared Types | Router |
| Router | Router | Rule Engine |
| Rule Engine | Destination Adapter | Device Manager |

| Module | Depends On | Provides To |
| --- | --- | --- |
| Dashboard | REST API, WebSocket | End Users |

## 10. Git Workflow

- **main:** Production stable branch.
- **develop:** Integration branch.
- **feature/:** Feature development branches.
- **Rules:** No direct push to main, Pull Requests only, Code Review required.

# Remaining sections :

## 1. Detailed Module Specifications

Detailed purpose, responsibilities, and lifecycle management for core components:

- **Gateway Core:** Orchestrates the message flow across the entire system.
- **Adapters (MQTT/HTTP/CoAP/WS):** Handles protocol-specific translation to and from the internal message model.
- **Device Manager:** Manages the registry, authentication, and device heartbeat status.
- **Rule Engine:** Executes conditional logic based on message content or metadata.

## 2. Sequence Diagrams

Runtime flows to be documented via Mermaid/Sequence notation:

- **Message Flow:** Device → Adapter → Internal Message → Router → Destination.
- **Device Management:** Registration and Authentication handshake.
- **Rule Execution:** Evaluation of IF-THEN triggers.

## 3. Component Diagram

High-level architectural view defining system boundaries including the Gateway, Dashboard, Database, and external Message Brokers, illustrating communication paths and inter-module dependencies.

## 4. Deployment Architecture

Infrastructure mapping for containerized services:

| Service | Port | Exposure |
|---|---|---|
| PostgreSQL | 5432 | Internal |
| Redis | 6379 | Internal |
| Mosquitto | 1883 | Public/Internal |
| Gateway Core | 3000 | Public |

## 5. Detailed Database Design (ER Diagram)

Comprehensive entity-relationship definitions covering Users, Devices, Rules, Messages, and Metrics, with enforced primary/foreign keys and indexes for telemetry optimization.

## 6. Complete API Specification

Full REST/WebSocket contract definitions including status codes, request/response payloads, and authentication requirements for management endpoints.

## 7. Internal Message Specification (Expanded)

Normalized schema field definitions including Type, Description, and validation rules for protocol-agnostic message handling.

## 8. Queue & Message Processing Design

Pipeline strategy for Incoming, Processing, Retry, and Dead-Letter Queues (DLQ) with strict acknowledgment protocols.

## 9. Security Architecture

Implementation of JWT lifecycle management, API keys, TLS transport security, and device-level authorization boundaries.

## 10. Error Handling & Recovery Strategy

Standardized retry policies with exponential backoff, timeout handling, and recovery patterns for broker failures.

## 11. Logging Strategy

Log levels (DEBUG, INFO, WARN, ERROR) with Correlation IDs to enable distributed tracing across asynchronous flows.

## 12. Monitoring & Health Checks

Definition of /health and /ready endpoints, and tracking of critical metrics such as message throughput and latency.

## 13. Plugin Architecture

Mechanism for the Init, Load, and Unload lifecycle of protocol adapters, supporting custom protocol registration.

## 14. Coding Standards & Project Conventions

Enforcement of TypeScript linting, naming conventions, and modular folder structures as defined in Phase 1.

## 15. Testing Strategy

Multi-tier testing framework comprising Unit, Integration, E2E flow simulation, and Load/Security testing.

## 16. Configuration Management (Expanded)

Secure environment variable management for secrets (JWT/DB) with environment-specific overrides.

## 17. Assumptions & Design Decisions

Architectural rationale for technology choices (NestJS, Redis) and documented system limitations or trade-offs.